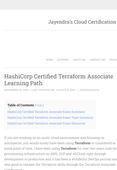

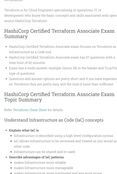

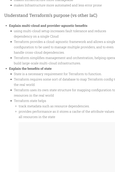

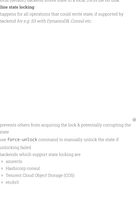

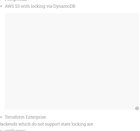

HashiCorp Certified Terraform Associate Learning Path

NOVEMBER 30, 2020 ~ LAST UPDATED ON : AUGUST 8, 2023 ~ JAYENDRAPATIL

Table of Contents [hide]

HashiCorp Certified Terraform Associate Exam Summary
HashiCorp Certified Terraform Associate Exam Topic Summary
HashiCorp Certified Terraform Associate Exam Resources

If you are working on an multi-cloud environment and focusing on automation, you would surely have been using **Terraform** or considered at some point of time. I have been using **Terraform** for over two years now for provisioning infrastructure on AWS, GCP and AliCloud right through development to production and it has been a wonderful DevOps journey and It was good to validate the Terraform skills through the Terraform Associate certification.

Terraform is for Cloud Engineers specializing in operations, IT, or development who know the basic concepts and skills associated with open source HashiCorp Terraform.

HashiCorp Certified Terraform Associate Exam Summary

- HashiCorp Certified Terraform Associate exam focuses on Terraform as a Infrastructure as a Code tool
- HashiCorp Certified Terraform Associate exam has 57 questions with a time limit of 60 minutes
- Exam has a multi answer, multiple choice, fill in the blanks and True/False type of questions
- Questions and answer options are pretty short and if you have experience on Terraform they are pretty easy and the time if more than sufficient.

HashiCorp Certified Terraform Associate Exam Topic Summary

Refer Terraform Cheat Sheet for details

Understand Infrastructure as Code (IaC) concepts

- **Explain what IaC is**
  - Infrastructure is described using a high-level configuration syntax
  - IaC allows Infrastructure to be versioned and treated as you would any other code.
  - Infrastructure can be shared and re-used.
- **Describe advantages of IaC patterns**
  - makes Infrastructure more reliable
  - makes Infrastructure more manageable
  - makes Infrastructure more automated and less error prone

Understand Terraform's purpose (vs other IaC)

- **Explain multi-cloud and provider-agnostic benefits**
  - using multi-cloud setup increases fault tolerance and reduces dependency on a single Cloud
  - Terraform provides a cloud-agnostic framework and allows a single configuration to be used to manage multiple providers, and to even handle cross-cloud dependencies.
  - Terraform simplifies management and orchestration, helping operators build large-scale multi-cloud infrastructures.
- **Explain the benefits of state**
  - State is a necessary requirement for Terraform to function.
  - Terraform requires some sort of database to map Terraform config to the real world.
  - Terraform uses its own state structure for mapping configuration to resources in the real world
  - Terraform state helps
    - track metadata such as resource dependencies.
    - provides performance as it stores a cache of the attribute values for all resources in the state
    - aids syncing when using in team with multiple users

Understand Terraform basics

- **Handle Terraform and provider installation and versioning**
  - Providers provide abstraction above the upstream API and is responsible for understanding API interactions and exposing resources.
  - Terraform configurations must declare which providers they require, so that Terraform can install and use them
  - Provider requirements are declared in a `required_providers` block.
- **Describe plugin based architecture**
  - Terraform relies on plugins called "providers" to interact with remote systems.
- **Demonstrate using multiple providers**
  - supports multiple provider instances using `alias` *for e.g. multiple aws providers with different region*
- **Describe how Terraform finds and fetches providers**
  - Terraform finds and installs providers when initializing a working directory. It can automatically download providers from a Terraform registry, or load them from a local mirror or cache.
  - Each Terraform module must declare which providers it requires, so that Terraform can install and use them.
- **Explain when to use and not use provisioners and when to use local-exec or remote-exec**
  - Terraform provides `local-exec` and `remote-exec` to execute tasks not provided by Terraform
    - local exec executes code on the machine running terraform
    - remote exec executes on the resource provisioned and supports `ssh` and `winrm`
  - Provisioners should only be used as a last resort.
  - are defined within the resource block.
  - support types – Create and Destroy
    - if creation time fails, resource is tainted if provisioning failed, by default. (next apply it will be re-created)
    - behavior can be overridden by setting the `on_failure` to `continue`, which means ignore and continue
    - for destroy, if it fails – resources are not removed

Use the Terraform CLI (outside of core workflow)

- **Given a scenario: choose when to use terraform fmt to format code**
  - `terraform fmt` helps format code to lint into a standard format. It usually aligns the spaces and matches the =
- **Given a scenario: choose when to use terraform taint to taint Terraform resources**
  - `terraform taint` marks a Terraform-managed resource as tainted, forcing it to be destroyed and recreated on the next apply.
  - will not modify infrastructure, but does modify the state file in order to mark a resource as tainted.
  - **Infrastructure and state are changed in next apply.**
  - can be used to taint a resource within a module
- **Given a scenario: choose when to use terraform import to import existing infrastructure into your Terraform state**
  - `terraform import` helps import already-existing external resources, not managed by Terraform, into Terraform state and allow it to manage those resources
  - Terraform is not able to auto-generate configurations for those imported modules, for now, and requires you to first write the resource definition in Terraform and then import this resource
- **Given a scenario: choose when to use terraform workspace to create workspaces**
  - Terraform workspace helps manage multiple distinct sets of infrastructure resources or environments with the same code.
  - state files for each workspace are stored in the directory `terraform.tfstate.d`
  - `terraform workspace new dev` creates a new workspace with name `dev` and switches to it as well
  - does not provide strong separation as it uses the same backend
- **Given a scenario: choose when to use terraform state to view Terraform state**
  - state helps keep track of the infrastructure Terraform manages
  - stored locally in the `terraform.tfstate`
  - recommended not to edit the state manually
  - Use `terraform state` command
    - `mv` – to move/rename modules
    - `rm` – to safely remove resource from the state. (destroy/retain like)
    - `pull` – to observe current remote state
    - `list` & `show` – to write/debug modules
- **Given a scenario: choose when to enable verbose logging and what the outcome/value is**
  - debugging can be controlled using `TF_LOG`, which can be configured for different levels TRACE, DEBUG, INFO, WARN or ERROR, with TRACE being the more verbose.
  - logs path can be controlled `TF_LOG_PATH` `TF_LOG` needs to be specified.

Interact with Terraform modules

- **Contrast module source options**
  - Terraform Module Registry allows you to browse, filter and search for modules
- **Interact with module inputs and outputs**
  - Input variables serve as parameters for a Terraform module, allowing aspects of the module to be customized without altering the module's own source code, and allowing modules to be shared between different configurations.
  - Resources defined in a module are encapsulated, so the calling module cannot access their attributes directly.
  - Child module can declare output values to selectively export certain values to be accessed by the calling module `module.module_name.output_value`
- **Describe variable scope within modules/child modules**
  - Modules are called from within other modules using `module` blocks
  - All modules require a `source` argument, which is a meta-argument defined by Terraform
  - To call a module means to include the contents of that module into the configuration with specific values for its input variables.
- **Discover modules from the public Terraform Module Registry**
  - Terraform Module Registry allows you to browse, filter and search for modules.
- **Defining module version**
  - must be on GitHub and must be a public repo, if using public registry.
  - must be named `terraform-<PROVIDER>-<NAME>`, where `<NAME>` reflects the type of infrastructure the module manages and `<PROVIDER>` is the main provider where it creates that infrastructure. *for e.g. terraform-google-vault or terraform-aws-ec2-instance.*
  - must maintain `x.y.z` tags for releases to identify module versions. and can optionally be prefixed with a v *for example, v1.0.4 and 0.9.2.* Tags that don't look like version numbers are ignored.
  - must maintain a Standard module structure, which allows the registry to inspect the module and generate documentation, track resource usage, parse submodules and examples, and more.

Navigate Terraform workflow

- **Describe Terraform workflow (Write -> Plan -> Create)**
  - Core Terraform workflow has three steps:
    - **Write** – Author infrastructure as code.
    - **Plan** – Preview changes before applying.
    - **Apply** – Provision reproducible infrastructure.
- **Initialize a Terraform working directory** `terraform init`
  - initializes a working directory containing Terraform configuration files.
  - performs backend initialization, modules and plugins installation.
  - plugins are downloaded in the sub-directory of the present working directory at the path of `.terraform/plugins`
  - does not delete the existing configuration or state
- **Validate a Terraform configuration** `terraform validate`
  - validates the configuration files in a directory, referring only to the configuration and not accessing any remote services such as remote state, provider APIs, etc.
  - verifies whether a configuration is syntactically valid and internally consistent, regardless of any provided variables or existing state.
  - useful for general verification of reusable modules, including the correctness of attribute names and value types.
- **Generate and review an execution plan for Terraform** `terraform plan`
  - `terraform plan` create a execution plan as it traverses each vertex and requests each provider using parallelism
  - calculates the difference between the last-known state and the current state and presents this difference as the output of the terraform plan operation to user in their terminal
  - does not modify the infrastructure or state.
  - allows a user to see which actions Terraform will perform prior to making any changes to reach the desired state
  - performs refresh for each resource and might hit rate limiting issues as it calls provider APIs
  - all resources refresh can be disabled or avoided using
    - `-refresh=false` or
    - `target=xxxx` or
    - break resources into different directories.
- **Execute changes to infrastructure with Terraform** `terraform apply`
  - will always ask for confirmation before executing unless passed the `-auto-approve` flag.

Discover more Cloud computing Google Cloud Platform Cloud Google Cloud Cloud Network

  - if a resource successfully creates but fails during provisioning, Terraform will error and mark the resource as "tainted". Terraform does not roll back the changes
- **Destroy Terraform managed infrastructure** `terraform destroy`

DISCLOSURE

This post may contain affiliate links, meaning when you click the links and make a purchase, we receive a commission.

Recent Posts

AWS Certified Machine Learning Engineer – Associate (MLA-C01) Exam Learning Path
AWS Certified AI Practitioner AIF-C01 Exam Learning Path
AWS Certified Data Engineer – Associate DEA-C01 Exam Learning Path
AWS Redshift Advanced
Amazon Athena

Categories

Select Category

    - will always ask for confirmation before executing unless passed the `-auto-approve` flag.

Implement and maintain state

- **Describe default local backend**
    - A "backend" in Terraform determines how state is loaded and how an operation such as `apply` is executed. This abstraction enables non-local file state storage, remote execution, etc.
    - determines how state is loaded and how an operation such as `apply` is executed
    - is responsible for storing state and providing an API for optional state locking
    - needs to be initialized
    - helps
        - collaboration and working as a team, with the state maintained remotely and state locking
        - can provide enhanced security for sensitive data
        - support remote operations
    - local (default) backend stores state in a local JSON file on disk
- **Outline state locking**
    - happens for all operations that could write state, if supported by backend *for e.g. S3 with DynamoDB, Consul etc.*
    - prevents others from acquiring the lock & potentially corrupting the state
    - use `force-unlock` command to manually unlock the state if unlocking failed
    - backends which support state locking are
        - azurerm
        - Hashicorp consul
        - Tencent Cloud Object Storage (COS)
        - etcdv3
        - Google Cloud Storage GCS
        - HTTP endpoints
        - Kubernetes Secret with locking done using a Lease resource
        - AliCloud Object Storage OSS with locking via TableStore
        - PostgreSQL
        - AWS S3 with locking via DynamoDB
        - Terraform Enterprise
    - Backends which do not support state locking are
        - artifactory
        - etcd
- **Handle backend authentication methods**
    - every remote backend support different authentication mechanism and can be configured with the backend configuration
- **Describe remote state storage mechanisms and supported standard backends**
    - remote backend stores state remotely like S3, OSS, GCS, Consul and support features like remote operation, state locking, encryption, versioning etc.
    - github is not a supported backend type.
- **Describe effect of Terraform refresh on state**
    - `terraform refresh`is used to reconcile the state Terraform knows about (via its state file) with the real-world infrastructure.
    - can be used to detect any drift from the last-known state, and to update the state file.
    - does not modify infrastructure but does modify the state file.
- **Describe backend block in configuration and best practices for partial configurations**
    - Backend configuration doesn't support interpolations.
    - supports partial configuration with remaining configuration arguments provided as part of the initialization process
    - if switching the backed for the first time setup, Terraform provides a migration option
- **Understand secret management in state files**
    - `terraform state` command is used for advanced state management
    - Terraform has no mechanism to redact or protect secrets that are returned via data sources, so secrets read via this provider will be persisted into the Terraform state, into any plan files, and in some cases in the console output produced while planning and applying.
    - can be protected accordingly either by using Vault and remote backends with encryption and proper access control

Read, generate, and modify configuration

- **Demonstrate use of variables and outputs**
    - Variables
        - serve as parameters for a Terraform module and
        - act like function arguments
        - count is a reserved word and cannot be used as variable name
    - Output
        - are like function return values.
        - can be marked `sensitive` which prevents showing its value in the list of outputs. However, they are stored in the state as plain text.
- **Describe secure secret injection best practice**
- **Understand the use of collection and structural types**
    - supports primitive data types of
        - `string`, `number` and `bool`
        - automatically convert `number` and `bool` values to `string` values
    - supports complex data types of
        - `list` – sequence of values identified by consecutive whole numbers starting with zero.
        - `map` – collection of values where each is identified by a string label
        - `set` – collection of unique values that do not have any secondary identifiers or ordering.
    - supports structural data types of
        - `object` – a collection of named attributes with their own type
        - `tuple` – a sequence of elements identified by consecutive whole numbers starting with zero, where each element has its own type.
- **Create and differentiate resource and data configuration**
    - Resources describe one or more infrastructure objects, such as virtual networks, instances, or higher-level components such as DNS records.
    - Data sources allow data to be fetched or computed for use elsewhere in Terraform configuration. Use of data sources allows a Terraform configuration to make use of information defined outside of Terraform, or defined by another separate Terraform configuration.
- **Use resource addressing and resource parameters to connect resources together**
- **Use Terraform built-in functions to write configuration**
    - `lookup` retrieves the value of a single element from a map, given its key. If the given key does not exist, a the given default value is returned instead. `lookup(map, key, default)`
    - `zipmap` constructs a map from a list of keys and a corresponding list of values. A map is denoted by { } whereas a list is donated by [ ] for e.g. `zipmap(["a", "b"], [1, 2])` results into `{"a" = 1, "b" = 2}`
- **Configure resource using a dynamic block**
    - `dynamic` acts much like a `for` expression, but produces nested blocks instead of a complex typed value. It iterates over a given complex value, and generates a nested block for each element of that complex value.
    - Overuse of dynamic block is not recommended as it makes the code hard to understand and debug
- **Describe built-in dependency management (order of execution based)**
    - Terraform analyses any expressions within a resource block to find references to other objects and treats those references as **implicit** ordering requirements when creating, updating, or destroying resources.
    - Explicit dependency can be defined using the `depends_on` attribute where dependencies between resources that are not visible
- support comments using #, // and /* */

Understand Terraform Cloud and Enterprise capabilities

- **Describe the benefits of Sentinel, registry, and workspaces**
    - Terraform Cloud provides private module registry for storing modules private to be used within the organization
- **Differentiate OSS and TFE workspaces**
- **Summarize features of Terraform Cloud**
    - Terraform Enterprise currently supports running under the following operating systems for a Clustered deployment:
        - Ubuntu 16.04.3 – 16.04.5 / 18.04
        - Red Hat Enterprise Linux 7.4 through 7.7
        - CentOS 7.4 – 7.7
        - Amazon Linux
        - Oracle Linux
        - Clusters currently don't support other Linux variants.
    - Terraform Enterprise install that is provisioned on a network that does not have Internet access is generally known as an **air-gapped** install.

HashiCorp Certified Terraform Associate Exam Resources

- Courses
    - KodeKloud – Terraform Associate Certification
- Practice tests
    - Braincert – HashiCorp Certified Terraform Associate Practice Exams
    - Whizlabs – Terraform Certification Exam Questions

POSTED IN TERRAFORM

ASSOCIATE CERTIFICATION HASHICORP TERRAFORM

< PREVIOUS
Terraform Cheat Sheet

NEXT >
Google Cloud Storage – Storage Classes

8 thoughts on "HashiCorp Certified Terraform Associate Learning Path"

Bilal Ashfaq
JANUARY 19, 2021 AT 10:35 PM

Thanks Jay for yet another excellent article!!! I am a long time follower of your posts! Very useful and informative posts.

jayendrapatil
JANUARY 20, 2021 AT 12:07 PM

Thanks for the feedback Bilal, glad its helping.

Jain
FEBRUARY 20, 2021 AT 6:07 PM

This is a great blog.

arun
MARCH 9, 2021 AT 6:33 PM

Can you recommend a course?

jayendrapatil
MARCH 11, 2021 AT 9:26 PM

Zeal Vora course on Terraform Udemy is a good one.

arun
MARCH 23, 2021 AT 11:14 AM

Thank you so much , I shall learn the course and take practice tests and then give it a try..thanks

you are inspiring many to do this course. 🙂

Coach32
DECEMBER 27, 2021 AT 12:46 PM

Thanks for this well put together article. It helped me understand Terraform IaC and study for the exam certification.

This alone can help pass the Terraform Associate Certification exam.

jayendrapatil
JANUARY 6, 2022 AT 8:12 AM

Glad it helped Coach32 …

Comments are closed.

Social media & sharing icons powered by UltimatelySocial